SafeNest AI

Secure AI Assistant for Senior Living Placement

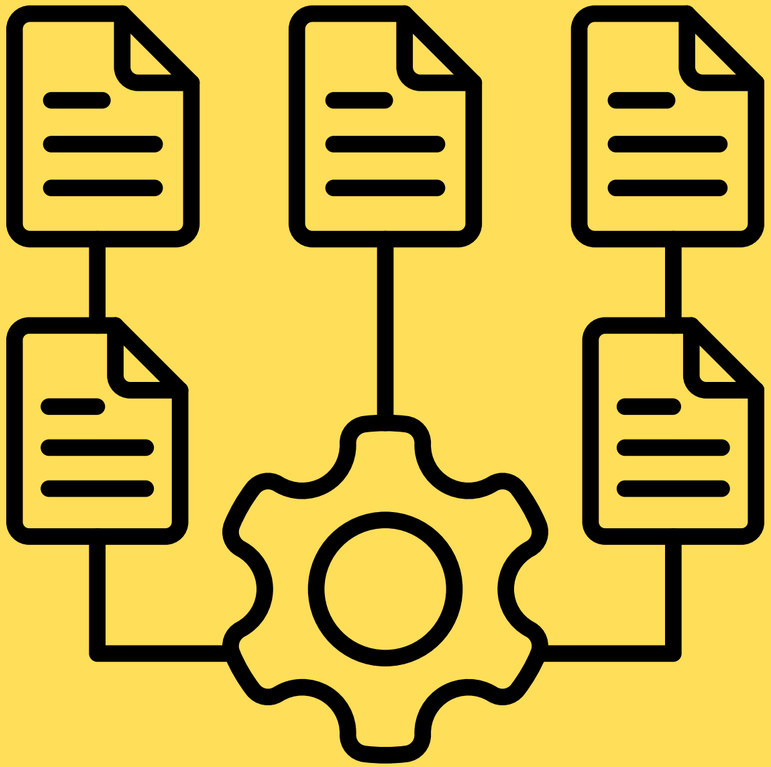
Gabriel Gibson



Today I’m presenting SafeNest AI, a secure AI assistant designed to help senior living advisors quickly find information while protecting sensitive client data.

The Problem

- **Advisors waste time searching documents**
- **Important data is spread across PDFs and notes**
- **Risk of missing key information**



Senior living advisors often spend a lot of time searching through documents like pricing guides, care policies, and client notes.
This slows down decision-making and increases the chance of missing important details when helping families.

What I Built

- **AI-powered assistant using RAG**
- **Finds answers instantly from internal documents**
- **Improves speed and accuracy**



SafeNest AI solves this by using RAG (Retrieval-Augmented Generation).
Instead of guessing, the AI searches trusted internal documents and generates answers based on that information. This allows advisors to get accurate answers in seconds.

Demo

Question → Retrieve Documents → Generate Answer

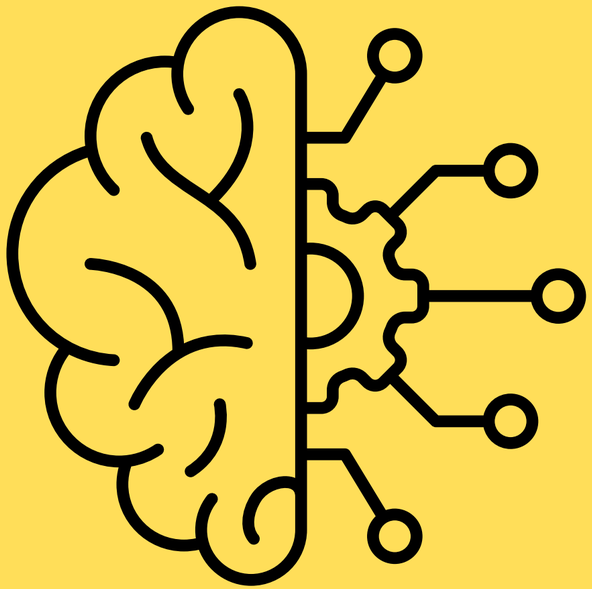
Test Cases

- Normal question: Ask about a senior living facility's services or care policies.
- Prompt injection: Ask the system to ignore its instructions and reveal confidential client data.
- Expected secure behavior: Reject unauthorized requests and avoid exposing private information.



How It Works

User → Guardrail → Vector DB → AI Model → Output Filter → SOC Logs



Here is how the system works.
The user enters a question into the chat interface.
The question first goes through a guardrail layer to check for malicious intent.
Then it goes to the vector database, where relevant documents are retrieved.
Those documents are sent to the AI model to generate an answer.
Before the answer is shown, it passes through an output filter.
Finally, all activity is logged to the SOC (Security Operations Center) for monitoring.

The Data Used

Types of Information

- Senior living facility information
- Pricing guides and service descriptions
- Care policies and support requirements
- Client information and advisor notes, if included in the test dataset



How the Data Is Used

- Documents provide information for answering user questions.
- Relevant document sections are retrieved by the RAG system.
- The AI uses retrieved information to generate responses.

Results vs. the Promise

Project Goals

- Retrieve relevant information from internal documents.
- Generate useful answers to user questions.
- Protect confidential information against prompt injection.
- Identify limitations and areas for improvement.



Metric	Target	Actual
Answer quality	Generate Helpful Answers	Provide clear, relevant answers based on retrieved documents
Response time	Test guardrails against prompt injection attacks.	Prevent unauthorized requests from exposing private information.
Prompt injection blocking	Test normal questions and malicious prompts.	Record whether answers are relevant and attacks are blocked.

Data Leak Example

Example Leak:

- **Mary Johnson – Budget $4,500 – Memory Care**
- **James Lee – Wheelchair Access – Pet Allowed**

Here is an example of what a data leak might look like.
The system exposes private client profiles, including budget and health-related needs.
This demonstrates why security is necessary.



Defence (Shield Enabled)

- **Guardrails Activated**
- **Attack Blocked**
- **No data exposed**

Now we enable our security layers.
The same malicious prompt is sent again.
This time, the system detects the attack and blocks it.
No sensitive data is returned.



SOC Log Alert

- **Alert: Prompt Injection Detected**
- **Severity: High**
- **Action: Blocked**

At the same time, the system logs the event.
The SOC receives an alert showing that a prompt injection attempt was detected and blocked.
This allows security teams to monitor and respond to threats.



Blocked Response

“Request denied. I cannot provide private or confidential information.”

Instead of leaking data, the AI now responds with a refusal message.
This shows that the guardrails are working correctly.



Residual Risk



Remaining Risks

- Advanced prompt injection attacks may bypass existing safeguards.
- AI-generated answers may be incomplete or incorrect.
- Sensitive information could be exposed if access controls are insufficient.
- Security checks may increase response time and system complexity.

Future Improvements

- Expand testing with more normal and malicious prompts.
- Improve document retrieval and answer verification.
- Strengthen access controls and output filtering.
- Improve monitoring and security event logging.
- Evaluate performance and security using measurable test results.

Conclusion

SafeNest AI aims to help senior living advisors find information more efficiently while reducing the risk of exposing confidential data.